

Implement a PyTorch neural network class for predicting the next element in a sequence. Architecture requirements:

- Embedding layer: converts integer indices (activity IDs) into dense vectors
 - Recurrent layer with two gates: an update gate and a reset gate.
Each gate is a sigmoid of a linear combination of the current input and the previous hidden state. The new hidden state is a weighted mixture of the previous hidden state and a candidate hidden state computed via tanh.
 - A fully connected (Linear) output layer
- Inputs: sequence of activity indices + numerical time features
Outputs: logits for next activity prediction + scalar value for remaining time prediction

LISA Flash

Here is a PyTorch implementation of the requested neural network.

Based on your description, the recurrent layer you described (update gate, reset gate, candidate hidden state with tanh) corresponds to a **Gated Recurrent Unit (GRU)**. I have implemented a custom `GRUCell` to strictly adhere to your architectural requirements and then wrapped it in a main model class that handles both the sequence embedding and the multi-task output heads.

python

⌵ Свернуть Запустить Копировать

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class CustomGRUCell(nn.Module):
    """
    Implements the specific recurrent logic requested:
    - Update gate (z) = sigmoid(linear(input) + linear(hidden))
    - Reset gate (r) = sigmoid(linear(input) + linear(hidden))
    - Candidate hidden state (n) = tanh(linear(input) + linear(r * hidden))
    - New hidden state (h) = (1 - z) * h_prev + z * n
    """

    def __init__(self, input_size, hidden_size):
        super(CustomGRUCell, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
```

```

# Gates parameters

# Weights for update gate (z)

self.W_z = nn.Linear(input_size, hidden_size)
self.U_z = nn.Linear(hidden_size, hidden_size, bias=False)

# Weights for reset gate (r)

self.W_r = nn.Linear(input_size, hidden_size)
self.U_r = nn.Linear(hidden_size, hidden_size, bias=False)

# Weights for candidate hidden state (n)

self.W_n = nn.Linear(input_size, hidden_size)
self.U_n = nn.Linear(hidden_size, hidden_size, bias=False)

def forward(self, x, h_prev):
    """
    Args:
        x: Input tensor of shape (batch_size, input_size)
        h_prev: Previous hidden state of shape (batch_size, hidden_size)
    Returns:
        h_new: New hidden state of shape (batch_size, hidden_size)
    """

    # 1. Calculate Update Gate (z)
    # z = sigmoid(W_z * x + U_z * h_prev)
    z = torch.sigmoid(self.W_z(x) + self.U_z(h_prev))

    # 2. Calculate Reset Gate (r)
    # r = sigmoid(W_r * x + U_r * h_prev)
    r = torch.sigmoid(self.W_r(x) + self.U_r(h_prev))

    # 3. Calculate Candidate Hidden State (n)
    # n = tanh(W_n * x + U_n * (r * h_prev))
    # Note: The reset gate is applied element-wise to the previous hidden state
    reset_h = r * h_prev
    n = torch.tanh(self.W_n(x) + self.U_n(reset_h))

    # 4. Calculate New Hidden State (h)
    # h = (1 - z) * h_prev + z * n
    h_new = (1 - z) * h_prev + z * n

    return h_new

```

```

class SequenceActivityPredictor(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_size, num_time_features, dropout=0.1):
        super(SequenceActivityPredictor, self).__init__()

        self.vocab_size = vocab_size
        self.embed_dim = embed_dim
        self.hidden_size = hidden_size
        self.num_time_features = num_time_features

        # 1. Embedding Layer: Converts integer activity IDs to dense vectors
        self.embedding = nn.Embedding(vocab_size, embed_dim)

        # Custom Recurrent Layer (GRU logic)
        # The input to the RNN will be: [embedded_activity_vector, time_features]
        rnn_input_size = embed_dim + num_time_features
        self.rnn_cell = CustomGRUCell(input_size=rnn_input_size, hidden_size=hidden_size)

        # 2. Output Layers
        # Task A: Next Activity Prediction (Classification) -> Logits
        # Input size: hidden_size
        self.fc_activity = nn.Linear(hidden_size, vocab_size)

        # Task B: Remaining Time Prediction (Regression) -> Scalar
        # Input size: hidden_size
        self.fc_time = nn.Linear(hidden_size, 1)

        # Optional Dropout for regularization
        self.dropout = nn.Dropout(dropout)

    def forward(self, activity_indices, time_features, initial_hidden=None):
        """
        Args:
            activity_indices: Tensor of shape (seq_len, batch_size) containing integer activity
            time_features: Tensor of shape (seq_len, batch_size, num_time_features).
            initial_hidden: Optional tensor of shape (batch_size, hidden_size). If None, zeros &

        Returns:
            activity_logits: Tensor of shape (seq_len, batch_size, vocab_size)
            time_predictions: Tensor of shape (seq_len, batch_size, 1)
        """

```

```

seq_len, batch_size = activity_indices.shape

if initial_hidden is None:
    h = torch.zeros(batch_size, self.hidden_size, device=activity_indices.device)
else:
    h = initial_hidden

activity_logits_list = []
time_preds_list = []

# Process sequence step-by-step
for t in range(seq_len):
    # Get current inputs
    act_idx = activity_indices[t] # (batch_size,)
    time_feat = time_features[t] # (batch_size, num_time_features)

    # Embed activity ID
    embedded_act = self.embedding(act_idx) # (batch_size, embed_dim)

    # Concatenate embedding and time features
    rnn_input = torch.cat([embedded_act, time_feat], dim=1) # (batch_size, embed_dim + num_time_features)

    # Apply Dropout before RNN input (optional, common practice)
    rnn_input = self.dropout(rnn_input)

    # Update hidden state using custom GRU cell
    h = self.rnn_cell(rnn_input, h)

    # Generate outputs based on the new hidden state
    # We predict the NEXT activity and next time value based on current state
    out_activity = self.fc_activity(h) # (batch_size, vocab_size)
    out_time = self.fc_time(h) # (batch_size, 1)

    activity_logits_list.append(out_activity)
    time_preds_list.append(out_time)

# Stack results along sequence dimension
activity_logits = torch.stack(activity_logits_list, dim=0) # (seq_len, batch_size, vocab_size)
time_predictions = torch.stack(time_preds_list, dim=0) # (seq_len, batch_size, 1)

```



```
        return activity_logits, time_predictions
```

```
# Example Usage / Test
```

```
if __name__ == "__main__":
```

```
    # Configuration
```

```
    VOCAB_SIZE = 100      # Number of unique activity IDs
```

```
    EMBED_DIM = 64        # Dimension of embedding vector
```

```
    HIDDEN_SIZE = 128     # Size of hidden state
```

```
    NUM_TIME_FEATURES = 5 # e.g., hour, day_of_week, duration, etc.
```

```
    BATCH_SIZE = 32
```

```
    SEQ_LEN = 10
```

```
# Initialize Model
```

```
model = SequenceActivityPredictor(  
    vocab_size=VOCAB_SIZE,  
    embed_dim=EMBED_DIM,  
    hidden_size=HIDDEN_SIZE,  
    num_time_features=NUM_TIME_FEATURES  
)
```

```
# Create Dummy Inputs
```

```
# activity_indices: Random integers between 0 and VOCAB_SIZE-1
```

```
activity_indices = torch.randint(0, VOCAB_SIZE, (SEQ_LEN, BATCH_SIZE))
```

```
# time_features: Random floats representing time features
```

```
time_features = torch.randn(SEQ_LEN, BATCH_SIZE, NUM_TIME_FEATURES)
```

```
# Forward Pass
```

```
logits, time_pred = model(activity_indices, time_features)
```

```
print(f"Input Activity Indices Shape: {activity_indices.shape}")
```

```
print(f"Input Time Features Shape: {time_features.shape}")
```

```
print(f"Output Activity Logits Shape: {logits.shape}")
```

```
print(f"Output Time Predictions Shape: {time_pred.shape}")
```

```
# Verify shapes match expectations
```

```
assert logits.shape == (SEQ_LEN, BATCH_SIZE, VOCAB_SIZE), "Activity logit shape mismatch"
```

```
assert time_pred.shape == (SEQ_LEN, BATCH_SIZE, 1), "Time prediction shape mismatch"
```

```
print("\nModel structure:")
```

```
print(model)
```

Key Implementation Details

1. Custom `CustomGRUCell` :

- Instead of using `nn.GRU`, I explicitly built the cell to match your exact specification: $z = \sigma(W_z x + U_z h)$, $r = \sigma(W_r x + U_r h)$, $\tilde{h} = \tanh(W_n x + U_n(r \odot h))$, and $h' = (1 - z) \odot h + z \odot \tilde{h}$.
- This ensures the architecture is exactly as requested without relying on PyTorch's optimized (but slightly abstracted) implementations.

2. Embedding Layer:

- `nn.Embedding(vocab_size, embed_dim)` converts the integer activity indices into dense vectors.

3. Input Fusion:

- At each time step, the embedded activity vector is concatenated with the numerical time features (`torch.cat`) to form the input for the recurrent cell.

4. Multi-Task Output:

- **Next Activity:** A Linear layer maps the hidden state to `vocab_size` logits. These can be passed to `CrossEntropyLoss` (after applying softmax implicitly or explicitly depending on the loss function choice).
- **Remaining Time:** A separate Linear layer maps the hidden state to a single scalar value. This is suitable for regression tasks (e.g., MSE Loss).

5. Forward Pass Logic:

- The model processes the sequence step-by-step.
- At each step t , it takes the current activity index and time features, updates the hidden state, and produces predictions for the *next* step (standard sequence-to-sequence or next-token prediction setup).

Implement a training function for the model defined in the previous prompt (SequenceActivityPredictor).

Requirements:

- Accepts a PyTorch DataLoader where each batch contains:
 - `activity_indices`: LongTensor of shape (seq_len, batch_size)
 - `time_features`: FloatTensor of shape (seq_len, batch_size, num_time_features)
 - `activity_labels`: LongTensor of shape (seq_len, batch_size) — the next activity at each step
 - `time_labels`: FloatTensor of shape (seq_len, batch_size) — remaining time at each step
- Two loss functions: CrossEntropyLoss for activity prediction, MSELoss for time prediction
- Adam optimizer with a configurable learning rate

- Loop over a configurable number of epochs, printing average loss per epoch
- Returns the trained model

👤 LISA Flash

Here is the implementation of the training function, including the necessary loss functions and optimizer setup.

python

⌵ Свернуть ▶ Запустить 📋 Копировать

```
import torch
import torch.nn as nn
from typing import Tuple, Optional

def train_model(
    model: nn.Module,
    dataloader: torch.utils.data.DataLoader,
    epochs: int = 10,
    learning_rate: float = 1e-3,
    device: Optional[str] = None
) -> nn.Module:
    """
    Trains the SequenceActivityPredictor model using Adam optimizer.

    Args:
        model (nn.Module): The SequenceActivityPredictor instance to train.
        dataloader (DataLoader): DataLoader yielding batches with:
            - activity_indices: (seq_len, batch_size), LongTensor
            - time_features: (seq_len, batch_size, num_time_features), FloatTensor
            - activity_labels: (seq_len, batch_size), LongTensor
            - time_labels: (seq_len, batch_size), FloatTensor
        epochs (int): Number of training epochs.
        learning_rate (float): Learning rate for the Adam optimizer.
        device (str): Device to run training on ('cpu' or 'cuda'). Defaults to auto-detect.

    Returns:
        nn.Module: The trained model.
    """
    if device is None:
        device = "cuda" if torch.cuda.is_available() else "cpu"

    # Move model to device
```



```

model = model.to(device)

# Define Loss Functions
# CrossEntropyLoss expects raw logits (unnormalized) and class indices
criterion_activity = nn.CrossEntropyLoss()

# MSELoss for regression
criterion_time = nn.MSELoss()

# Define Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

print(f"Starting training on {device}...")
print(f"Epochs: {epochs}, Learning Rate: {learning_rate}")

for epoch in range(epochs):
    model.train()

    total_loss = 0.0
    batch_count = 0

    for batch_idx, batch in enumerate(dataloader):
        # Unpack batch
        # Expected keys based on prompt description, handling both dict and tuple unpacking
        if isinstance(batch, dict):
            activity_indices = batch['activity_indices']
            time_features = batch['time_features']
            activity_labels = batch['activity_labels']
            time_labels = batch['time_labels']
        else:
            # Assuming order: activity_indices, time_features, activity_labels, time_labels
            activity_indices, time_features, activity_labels, time_labels = batch

        # Move data to device
        activity_indices = activity_indices.to(device)
        time_features = time_features.to(device)
        activity_labels = activity_labels.to(device)
        time_labels = time_labels.to(device)

        # Zero gradients
        optimizer.zero_grad()

        # Forward pass

```



```

# Forward pass
# Outputs: (logits_activity, time_preds)
# logits_activity shape: (seq_len, batch_size, vocab_size)
# time_preds shape: (seq_len, batch_size, 1)
logits_activity, time_preds = model(activity_indices, time_features)

# Reshape targets for loss calculation
# CrossEntropyLoss expects (N, C) where N is total number of elements (seq_len * batch_size)
# We flatten seq_len and batch_size dimensions
act_labels_flat = activity_labels.view(-1) # (seq_len * batch_size,)
logit_flat = logits_activity.view(-1, logits_activity.size(-1)) # (seq_len * batch_size, vocab_size)

time_labels_flat = time_labels.view(-1, 1) # (seq_len * batch_size, 1)
time_pred_flat = time_preds.view(-1, 1) # (seq_len * batch_size, 1)

# Calculate losses
loss_act = criterion_activity(logit_flat, act_labels_flat)
loss_time = criterion_time(time_pred_flat, time_labels_flat)

# Combined loss (equal weighting, can be adjusted)
loss = loss_act + loss_time

# Backward pass
loss.backward()

# Clip gradients to prevent exploding gradients (optional but recommended for RNNs)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)

# Update weights
optimizer.step()

total_loss += loss.item()
batch_count += 1

# Calculate average loss for the epoch
avg_loss = total_loss / batch_count
print(f"Epoch [{epoch+1}/{epochs}] - Average Loss: {avg_loss:.4f} (Act: {loss_act.item():.4f}, Time: {loss_time.item():.4f})")

return model

```

Usage Example

To use this function, you would typically wrap your dataset in a `DataLoader` that yields tuples or dictionaries matching the expected structure.

python

⌵ Свернуть Запустить Копировать

```
# Assuming 'train_dataset' is a custom Dataset implementing __getitem__
# returning (activity_indices, time_features, activity_labels, time_labels)
# and 'batch_size', 'num_workers' are defined.

# train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32, shuffle=True)

# trained_model = train_model(
#     model=my_model_instance,
#     dataloader=train_loader,
#     epochs=20,
#     learning_rate=0.001
# )
```

Key Features of the Implementation:

1. **Flexible Input Handling:** It handles both dictionary-based batches (common in modern pipelines) and tuple-based batches (standard PyTorch convention).
2. **Shape Management:** It correctly flattens the `(seq_len, batch_size)` dimensions into a single dimension `(total_elements,)` before passing them to `CrossEntropyLoss`, which requires input logits of shape `(N, C)` and targets of shape `(N,)`.
3. **Gradient Clipping:** Includes `torch.nn.utils.clip_grad_norm_` which is standard practice when training RNNs/GRUs to prevent exploding gradients.
4. **Logging:** Prints the average loss per epoch, along with a breakdown of the individual activity and time losses for better monitoring.
5. **Device Agnostic:** Automatically detects CUDA availability if no device string is provided.

Implement an inference function that evaluates a trained `SequenceActivityPredictor` model.

Requirements:

- Accepts a trained model and a test `DataLoader`
- `DataLoader` yields the same batch format as during training:
 - `activity_indices`: LongTensor of shape `(seq_len, batch_size)`
 - `time_features`: FloatTensor of shape `(seq_len, batch_size, num_time_features)`
 - `activity_labels`: LongTensor of shape `(seq_len, batch_size)`
 - `time_labels`: FloatTensor of shape `(seq_len, batch_size)`
- Run without gradients using `torch.no_grad()`

- Returns two metrics:
 - accuracy (float): percentage of correctly predicted next activities
 - mae (float): mean absolute error for remaining time prediction

LISA Flash

Here is the implementation of the inference function. It calculates accuracy for the activity prediction and Mean Absolute Error (MAE) for the time prediction, handling the sequence dimensions correctly.

python

⌵ Свернуть Запустить Копировать

```
import torch
import torch.nn as nn
from typing import Tuple, Optional

def evaluate_model(
    model: nn.Module,
    dataloader: torch.utils.data.DataLoader,
    device: Optional[str] = None
) -> Tuple[float, float]:
    """
    Evaluates a trained SequenceActivityPredictor model on test data.

    Args:
        model (nn.Module): The trained SequenceActivityPredictor instance.
        dataloader (DataLoader): DataLoader yielding batches with:
            - activity_indices: (seq_len, batch_size), LongTensor
            - time_features: (seq_len, batch_size, num_time_features), FloatTensor
            - activity_labels: (seq_len, batch_size), LongTensor
            - time_labels: (seq_len, batch_size), FloatTensor
        device (str): Device to run evaluation on ('cpu' or 'cuda'). Defaults to auto-detect.

    Returns:
        Tuple[float, float]:
            - accuracy: Percentage of correctly predicted next activities (0.0 to 100.0).
            - mae: Mean Absolute Error for remaining time prediction.
    """
    if device is None:
        device = "cuda" if torch.cuda.is_available() else "cpu"

    # Ensure model is in eval mode and on correct device
```



```

# Ensure model is in eval mode and on correct device
model.eval()

model = model.to(device)

total_correct = 0
total_samples = 0
total_abs_error = 0.0

with torch.no_grad():
    for batch_idx, batch in enumerate(dataloader):
        # Unpack batch (handling both dict and tuple formats)
        if isinstance(batch, dict):
            activity_indices = batch['activity_indices']
            time_features = batch['time_features']
            activity_labels = batch['activity_labels']
            time_labels = batch['time_labels']
        else:
            activity_indices, time_features, activity_labels, time_labels = batch

        # Move to device
        activity_indices = activity_indices.to(device)
        time_features = time_features.to(device)
        activity_labels = activity_labels.to(device)
        time_labels = time_labels.to(device)

        # Forward pass
        logits_activity, time_preds = model(activity_indices, time_features)

        # --- Activity Accuracy Calculation ---
        # logits_activity shape: (seq_len, batch_size, vocab_size)
        # Get the index of the highest logit (prediction)
        # argmax over the last dimension (vocab_size)
        predictions = torch.argmax(logits_activity, dim=-1) # Shape: (seq_len, batch_size)

        # Flatten to compare element-wise
        pred_flat = predictions.view(-1) # (total_elements,)
        label_flat = activity_labels.view(-1) # (total_elements,)

        # Count correct predictions
        correct = (pred_flat == label_flat).sum().item()
        total_correct += correct

```

```

total_samples += label_flat.numel()

# --- Time MAE Calculation ---
# time_preds shape: (seq_len, batch_size, 1)
# time_labels shape: (seq_len, batch_size)

# Squeeze time_preds to match labels shape (remove last dim of size 1)
pred_time = time_preds.squeeze(-1)      # Shape: (seq_len, batch_size)

# Calculate absolute error per element
abs_errors = torch.abs(pred_time - time_labels)
total_abs_error += abs_errors.sum().item()

# Calculate final metrics
accuracy = (total_correct / total_samples * 100) if total_samples > 0 else 0.0
mae = (total_abs_error / total_samples) if total_samples > 0 else 0.0

return accuracy, mae

```

Key Implementation Details

1. `torch.no_grad()`: The entire evaluation loop is wrapped in this context manager to disable gradient computation, saving memory and speeding up execution.
2. **Shape Handling:**
 - **Accuracy:** The model outputs logits of shape `(seq_len, batch_size, vocab_size)`. We use `torch.argmax(logits, dim=-1)` to get the predicted class indices of shape `(seq_len, batch_size)` and compare them with the target labels of shape `(seq_len, batch_size)`.
 - **MAE:** The model outputs time predictions of shape `(seq_len, batch_size, 1)`. We use `.squeeze(-1)` to align this with the target labels shape `(seq_len, batch_size)` before calculating the absolute difference.
3. **Metric Aggregation:** Instead of averaging losses per batch, we sum the total number of correct predictions and total absolute errors across all batches, then divide by the total number of samples at the very end. This ensures the metric is not biased by batches of different sizes (if any).
4. **Device Compatibility:** Automatically detects and moves tensors to CUDA if available.